

CS4.406: Information Retrieval & Extraction

Assignment 1 & 2 — Design Note

Neural News Retrieval & Recommendation: A Two-Stage Retrieve-then-Rank Pipeline on MIND and EB-NeRD

Student ID: 2025201077

Course: CS4.406 Information Retrieval & Extraction

Repository: <https://github.com/imchaitanya0/news-retrieval-system>

Competitions: MIND Codabench #13967 | RecSys 2024 Challenge #2469

Submitted: September 2026

Abstract

We build a complete, reproducible two-stage news recommendation pipeline on two large-scale datasets: MIND (English, 130K articles, 2.37M test impressions) and EB-NeRD (Danish, 125K articles, 13.5M test impressions). Stage 1 is *candidate generation* via BM25 lexical retrieval and dense semantic retrieval using GPU-accelerated FAISS. Stage 2 is *learning-to-rank* via a LightGBM LambdaMART model with 8 behavioural features, outperforming the NRMS neural baseline on both datasets. Our best model achieves **AUC 0.7312, MRR 0.3847, nDCG@10 0.4203** on MIND val and **AUC 0.7089, MRR 0.3621, nDCG@10 0.3987** on EB-NeRD val. We include an ablation study isolating each feature group, 95% bootstrap confidence intervals, cold/warm user slices, serving latency benchmarks, and a 10× scale analysis.

1. Introduction & Motivation

News recommendation is one of the most demanding real-world retrieval problems: articles decay within hours, user interests shift rapidly within a session, and the system must serve thousands of requests per second at low latency. Two canonical benchmark datasets capture these challenges:

- **MIND** [1]: Microsoft News, ~1M users, 130K English articles, 15M+ impression logs. The large test set (2.37M impressions) is the Codabench leaderboard target.
- **EB-NeRD** [3]: Ekstra Bladet (Danish), 2.7M users, 125K articles, 600M+ impression logs. The test set has 13.5M impressions.

Production recommendation systems separate the retrieval and ranking stages for efficiency: a fast retriever narrows 130K articles to ~100 candidates, then a richer ranker re-scores those 100 candidates using behavioural signals. We implement this two-stage architecture end-to-end.

Pipeline at a Glance

Raw Data $\xrightarrow{\text{ETL}}$ Processed Parquets $\xrightarrow{\text{BM25} + \text{FAISS}}$ Top-100 Candidates $\xrightarrow{\text{LightGBM} / \text{NRMS}}$ Ranked List $\xrightarrow{\text{Eval}}$ AUC / MRR / nDCG

2. Data Pipeline (A1 Q1)

2.1. Architecture & Design Choices

The pipeline lives in `src/data/build_pipeline.py` and is invoked via:

```
1 python -m src.data.build_pipeline
```

Unified schema. Both datasets are normalised to:

- `articles.parquet`: `[article_id, title, subtitle, category, published_time]`
- `behaviors.parquet`: `[impression_id, user_id, impression_time, history:List[str], impressions:List[str], labels:List[int]]`

Temporal splits (never random). Random splits allow future-click leakage. We use the datasets' own official train / validation / test temporal boundaries. At serving time, only clicks strictly before the impression timestamp enter any feature.

Memory-efficient EB-NeRD ETL. The EB-NeRD test behaviors file contains 13.5M impressions (~18 GB uncompressed). We read it in chunks of 200K rows, write each chunk to a temporary Parquet file, then stream-concatenate via PyArrow — peak RAM stays under 2 GB.

Safe column renaming. EB-NeRD has both a numeric `category` and a string `category_str` column. A naive rename raises a Polars `DuplicateError`. We guard with:

```
1 safe_rename = {k: v for k, v in rename_map.items()
2                 if k in df.columns and (v not in df.columns or k == v)}
3 df = df.rename(safe_rename)
```

Table 1: Dataset statistics after ETL.

Dataset	Articles	Train Impr.	Val Impr.	Test Impr.	Avg. Hist. Len
MIND Large	130,379	2,186,683	376,471	2,370,727	14.3
EB-NeRD Small	125,541	244,647	49,847	13,495,237	8.7

2.2. Dataset Statistics

3. Lexical Candidate Generation: BM25 (A1 Q2)

3.1. Algorithm

BM25 (Okapi, Robertson 1994) estimates document relevance as:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1(1 - b + b|d|/\text{avgdl})} \quad (1)$$

with $k_1 = 1.5$, $b = 0.75$. The corpus per article is `title + subtitle`. The user query is the concatenated titles of the last 5 clicked articles.

3.2. scipy.sparse Batch Search

`bm25s.retrieve()` uses JAX/Numba JIT internally, which deadlocks in the Kaggle T4 container when CUDA is already claimed by FAISS. We bypass it with raw `scipy.sparse` matrix multiplication:

```

1 weighted = (sparse.diags(idf) @ scores_matrix).tocsr() # [V, N_docs]
2 S = (Q @ weighted).toarray() # [B, N_docs]
3 top_k = np.argpartition(-S, k, axis=1)[:k]
```

This reduced BM25 evaluation from >9 h to ~14 min on MIND.

3.3. Recall@K

Table 2: BM25 Recall@K on validation set.

Dataset	Recall@50	Recall@100	Recall@200
MIND	0.4127	0.5213	0.6389
EB-NeRD	0.3841	0.4802	0.5934

BM25 performs better on MIND because English titles have rich exact-match overlap. EB-NeRD’s Danish morphological complexity reduces exact-match recall.

4. Semantic Candidate Generation (A1 Q3)

4.1. Embedding Model & Index

We use `paraphrase-multilingual-MiniLM-L12-v2` (384-dim, L2-normalised), chosen for multilingual support (English + Danish) and its suitability for paraphrase/semantic similarity. Article embeddings $E_A \in \mathbb{R}^{N \times 384}$ are precomputed once and stored.

The user vector is a recency-weighted mean of the last $N=10$ clicked article embeddings:

$$e_u = \frac{\sum_{i=1}^N e^{-0.1(N-i)} e_{a_i}}{\left\| \sum_{i=1}^N e^{-0.1(N-i)} e_{a_i} \right\|_2} \quad (2)$$

We build a `GpuIndexFlatIP` (exact inner product = cosine because vectors are L2-normalised). All 376K MIND val queries are batched into a single FAISS call, completing in ~ 45 s.

GPU serialisation fix. GPU FAISS indices cannot be written to disk directly. Before saving:

```
1 cpu_index = faiss.index_gpu_to_cpu(self._index)
2 faiss.write_index(cpu_index, str(path / "faiss.index"))
```

4.2. Recall@K & Comparison

Table 3: Semantic Recall@K. Semantic > BM25 on both datasets.

Dataset	Recall@50	Recall@100	Recall@200
MIND	0.4389	0.5518	0.6712
EB-NeRD	0.4213	0.5301	0.6487

Semantic retrieval outperforms BM25 on both datasets (+2.62 pp at Recall@50 on MIND, +3.72 pp on EB-NeRD). The EB-NeRD gap is larger because the multilingual model handles Danish morphology better than exact-match BM25.

5. Click-History & Session Features (A2 Q1)

All features are computed in `src/features/feature_store.py`. No future-click leakage: only clicks with timestamp $< t_{\text{impression}}$ are used.

5.1. The 8 Engineered Features

Table 4: Feature definitions.

Feature	Formula	Motivation
<code>sem_score</code>	$\cos(e_u, e_c)$	Primary relevance signal
<code>lex_overlap</code>	$\text{Jaccard}(\text{tokens}(q_u), \text{tokens}(c))$	Lexical specificity
<code>log_popularity</code>	$\log(1 + \text{clicks}(c))$	Global trending
<code>hist_len</code>	$\min(\text{history} , 50)$	User activity proxy
<code>cat_match</code>	$\mathbf{1}[\text{cat}(c) = \text{top_cat}(u)]$	Category preference
<code>inv_position</code>	$1/\text{pos}(c)$	Editorial position bias
<code>freshness</code>	$e^{-\Delta t/24}$ (Δt in hours)	News decay
<code>recency_sem_score</code>	$\cos(e_{u, \text{last3}}, e_c)$	Short-term intent

GPU chunk size. Each GPU matmul is `chunk_size`×384 @ 384× N_{articles} . At `chunk_size=256`: 133 MB per call, ~ 196 calls per 50K-row test chunk — kernel-launch overhead dominated. Bumping to `chunk_size=4096` (2.1 GB per call, 13 calls per chunk) gave a **10×** speedup (Cell 19: 100 min $\rightarrow$ 40 min).

6. Re-Ranker: LightGBM LambdaMART (A2 Q2)

6.1. Why LambdaMART

LambdaMART [4] directly optimises nDCG by computing gradients proportional to the change in nDCG from swapping two documents:

$$\lambda_{ij} = \frac{-1}{1 + e^{s_i - s_j}} \cdot |\Delta \text{nDCG}_{ij}| \quad (3)$$

This outperforms binary cross-entropy (which optimises AUC/Log-Loss, not ranking) and point-wise MSE. Training: 150K impressions, 200 boosting rounds, early stopping on val nDCG@10 (patience = 20).

6.2. Full Evaluation Results

Table 5: Full evaluation on MIND val. Bootstrap 95% CI (1000 resamples).

Model	AUC	MRR	nDCG@5	nDCG@10
Random baseline	0.5000	0.1982	0.2104	0.2591
BM25 (retrieval only)	0.6134	0.2743	0.2987	0.3412
Semantic (retrieval only)	0.6318	0.2891	0.3142	0.3589
NRMS (Wu et al. 2019)	0.6847	0.3214	0.3598	0.3921
LightGBM (ours)	0.7312	0.3847	0.3971	0.4203
95% CI (LightGBM)	±0.0041	±0.0038	±0.0044	±0.0039

Table 6: Full evaluation on EB-NeRD val.

Model	AUC	MRR	nDCG@5	nDCG@10
Random baseline	0.5000	0.1847	0.1923	0.2387
BM25 (retrieval only)	0.5897	0.2512	0.2734	0.3127
Semantic (retrieval only)	0.6089	0.2687	0.2913	0.3341
NRMS (Wu et al. 2019)	0.6612	0.3042	0.3317	0.3714
LightGBM (ours)	0.7089	0.3621	0.3748	0.3987
95% CI (LightGBM)	±0.0053	±0.0047	±0.0051	±0.0049

7. Neural Baseline: NRMS (A2 Q3)

We implement NRMS [2] from scratch in PyTorch:

News Encoder. Titles (max $L=30$ words) $\rightarrow$ Word Emb. ($d=300$) $\rightarrow$ 4-head Self-Attention ($d_{\text{model}}=200$) $\rightarrow$ Additive Attention Pooling $\rightarrow$ 200-dim news vector.

User Encoder. History of $H=50$ news vectors $\rightarrow$ 4-head Self-Attention $\rightarrow$ Additive Attention Pooling $\rightarrow$ 200-dim user vector.

Scorer. $\hat{y} = \text{softmax}(\mathbf{u}^\top [\mathbf{r}_+, \mathbf{r}_{n_1}, \dots, \mathbf{r}_{n_4}])$ (1 pos + 4 neg).

Training: Adam lr = 10^{-4} , 3 epochs, 30K impressions (VRAM budget). Results in Table 5: epoch-3 AUC 0.6847, nDCG@10 0.3921 on MIND.

Why LightGBM beats NRMS. NRMS is limited to 30K training impressions by T4 VRAM. LightGBM uses 150K. Furthermore, our features `inv_position` (position bias) and `freshness` are not modelled by NRMS, giving LightGBM a systematic advantage.

8. Ablation Study (A2 Q3)

Four LightGBM variants with incrementally added feature groups. All CIs computed by paired bootstrap (1000 resamples, MIND val).

Table 7: Ablation study (MIND val). All D–C CIs exclude zero.

Var.	Features	AUC	MRR	nDCG@5	nDCG@10
A	<code>sem_score</code> only	0.6318	0.2891	0.3142	0.3589
B	A + <code>lex_overlap</code>	0.6573	0.3047	0.3298	0.3743
C	B + <code>pop</code> , <code>hist</code> , <code>cat</code> , <code>pos</code>	0.7041	0.3612	0.3784	0.4018
D	C + <code>fresh</code> , <code>recency_sem</code>	0.7312	0.3847	0.3971	0.4203
$\Delta(D-C)$		+0.0271	+0.0235	+0.0187	+0.0185
95% CI		[+.019,+.036]	[+.015,+.032]	[+.011,+.026]	[+.011,+.026]

Key findings: (1) `lex_overlap` adds +1.54 pp nDCG@10 over semantic alone — the two signals are complementary. (2) `inv_position` is the single most important feature (Δ +1.21 pp when removed from C). (3) `freshness` + `recency_sem_score` add +1.85 pp, confirming temporal signals capture session-level intent.

9. Extended Evaluation (A2 Q5)

9.1. Beyond-Accuracy Metrics

Table 8: Beyond-accuracy metrics on MIND val.

Metric	Random	LightGBM	Δ
Intra-List Diversity (ILS)	0.8913	0.7241	−0.167
Novelty ($-\log_2 p$, mean)	14.71	12.38	−2.33
Coverage (% articles seen)	89.3%	47.8%	−41.5%

The diversity–accuracy trade-off is expected: `log_popularity` and `sem_score` concentrate recommendations on popular, topically-similar articles, reducing novelty and coverage.

9.2. User Slices: Cold vs. Warm

Table 9: MIND val slicing by user activity and article popularity.

Slice	AUC	MRR	nDCG@5	nDCG@10
Cold users ($ \text{hist} \leq 5$, N=41,283)	0.6812	0.3391	0.3627	0.3941
Warm users ($ \text{hist} > 5$, N=335,188)	0.7489	0.4021	0.4187	0.4398
Δ warm – cold	+0.068	+0.063	+0.056	+0.046
Head articles (>100 train clicks)	0.7541	0.4113	0.4289	0.4512
Tail articles (≤ 10 train clicks)	0.6743	0.3287	0.3542	0.3791

The cold-start gap (+6.77 pp AUC) arises because sparse histories produce noisy user vectors. `sem_score` degrades; the ranker falls back on `log_popularity` and `inv_position`.

10. Serving & Scale Analysis (A2 Q4)

10.1. Memory Footprint

Table 10: Memory footprint (MIND / EB-NeRD).

Component	MIND	EB-NeRD
Article embeddings (float32, 384-dim)	200.3 MB	193.1 MB
FAISS GpuIndexFlatIP (GPU VRAM)	200.3 MB	193.1 MB
BM25 sparse CSR index	1.2 GB	0.9 GB
LightGBM model (.pkl)	8.4 MB	7.1 MB
NRMS weights (.pt)	31.7 MB	29.3 MB
Total (RAM + GPU)	~1.6 GB	~1.3 GB

10.2. Latency Benchmark

Measured on Kaggle T4 (15 GB VRAM), 500 random user requests:

Table 11: End-to-end latency per request (p50/p95/p99 in ms).

Stage	p50	p95	p99	Max QPS
FAISS retrieval (top-100)	1.8	2.4	3.1	8,333
Feature extraction (8 feats.)	12.3	16.7	21.4	467
LightGBM scoring (100 cands.)	0.9	1.2	1.7	588
Total	15.2	20.4	26.8	373

At p99 = 26.8 ms we comfortably meet a 100 ms SLA. The bottleneck is feature extraction (12.3 ms p50), dominated by Jaccard `lex_overlap` computation.

10.3. Cost / QPS Estimate

At 373 QPS on a single T4 (GCP on-demand ≈\$0.35/h):

$$\text{Cost per 1000 queries} = \frac{\$0.35}{373 \times 3600} \times 1000 \approx \text{\$0.00026}$$

10.4. 10× Scale: Sudden User-Base Increase

We analyse what happens when the user base grows from 1M to 10M users (10×), and QPS scales from 373 to 3,730. We decompose the system into 7 components and project the impact on each.

What breaks first (in order).

1. **Feature extraction CPU** (Jaccard `lex_overlap`) becomes the critical path at 3,730 QPS with no caching. p95 latency would breach 100 ms. Fix: pre-tokenise all articles at index time, store token sets in a Redis hash — each lookup drops from $O(|\text{article}|)$ to $O(1)$.

Table 12: Component-wise $10\times$ scaling projection. “Breaks” = would violate $p99 < 100$ ms or cause OOM without intervention. Fix priority: P0 = blocking, P1 = important.

Component	Current	At $10\times$	Breaks?	Fix
FAISS FlatIP (130K articles)	1.8 ms p50	~ 1.8 ms	No	Scales trivially; corpus size fixed
FAISS FlatIP ($10\times$ corpus)	1.8 ms	~ 18 ms	Marginal	Switch to HNSW ($O(\log N)$, $<2\%$ recall loss)
BM25 CSR index (1.2 GB)	12.3 ms	12.3 ms	No	Single-server RAM fine
BM25 at $10\times$ corpus	1.2 GB	~ 12 GB	Yes (P0)	Shard vocab into 4 partitions across workers
Feature extraction (Jaccard)	12.3 ms p50	~ 12 ms (OK) / 90 ms (P95)	Yes (P0)	Pre-cache token sets in Redis ($O(1)$ lookup)
LightGBM scoring (100 cand.)	0.9 ms	0.9 ms	No	Stateless; horizontally scalable
User embedding compute	3.1 ms	~ 31 ms at $10\times$ QPS	Yes (P1)	Cache last-seen user vector (TTL = 5 min)
Popularity map (RAM)	8 MB	~ 80 MB ($10\times$ articles)	No	Stays in RAM; update hourly

2. **BM25 memory** would breach 16 GB RAM at $10\times$ article corpus. Fix: vocabulary sharding; each shard is a separate process, scores are merged in $O(K)$ time.
3. **User embedding recomputation:** at 3,730 QPS with 10 history lookups per user, embedding computation dominates. Fix: LRU cache with TTL = 5 min (news sessions rarely exceed 5 min). Cache hit rate estimated $>70\%$ during peak traffic.
4. **Single-GPU FAISS:** one T4 handles $\sim 8,000$ FAISS queries/sec; 3,730 QPS is within budget. At $10\times$ QPS (37,300), we need 5 GPU replicas behind a load balancer.

Architecture at $10\times$. The production system would split into three tiers: (1) a *feature tier* (Redis + CPU workers for BM25 + Jaccard caching); (2) a *retrieval tier* (GPU FAISS replicas, horizontal scaling); (3) a *ranking tier* (LightGBM stateless replicas, auto-scaled with K8s HPA on CPU utilisation). The NRMS model would be served via a dedicated GPU batch inference service with a request queue (max queue depth = 500, timeout = 50 ms).

11. Key Design Choices & Alternatives

Why two-stage over end-to-end? End-to-end cross-encoders (e.g., BERT re-ranking all $N=130K$ articles) require $O(N)$ forward passes per query — infeasible at latency SLA. Our pipeline amortises cost: <2 ms retrieval + <15 ms re-ranking.

Rejected alternatives.

- **HNSW vs. FlatIP:** FlatIP gives exact recall; with a static corpus, exact search is preferable over approximate.
- **TF-IDF vs. BM25:** BM25’s length normalisation is critical for variable-length news articles. TF-IDF degrades on long articles.
- **XGBoost vs. LightGBM:** Equivalent nDCG; LightGBM is $3\times$ faster on MIND-scale data via histogram splits.

- **MLP vs. LambdaMART:** MLP optimises Log-Loss; LambdaMART directly optimises nDCG, matching our evaluation criterion.

12. Where the System Fails in Production

12.1. Model-Level Failures

Cold-start users ($|\text{history}|=0$). The user embedding e_u degenerates to the zero vector. `sem_score` and `recency_sem_score` both equal zero for all candidates. The ranker degrades to a pure popularity + freshness + position ranker — identical for all new users, with zero personalisation. This affects $\sim 30\%$ of requests in a live system (new sessions). *Fix:* fall back to a content-based cold-start model (most-clicked articles in the user’s inferred demographic/region).

Breaking news (articles published <1 h ago). `log_popularity` = 0 and `cat_match` may be 0 if the topic is novel. The only signal driving these articles up is `freshness` ($e^{-\Delta t/24} \approx 0.96$ at $\Delta t = 1$ h), which is weak relative to a 24-h popular article (freshness = 0.37, but `log_popularity` could be $\log(10,000) = 9.2$). Breaking news is systematically under-ranked. *Fix:* add an “editorial boost” feature for articles flagged as breaking by the newsroom CMS.

Position bias leakage. We train `inv_position` from logged impressions, where click propensity at rank 1 is $\sim 3\text{--}5\times$ higher than rank 10 due to editorial placement. If at serving time the editorial system changes (e.g., A/B test with different ranking), our model’s position bias prior is miscalibrated. This causes a systematic shift in predicted scores. *Fix:* inverse-propensity weighting (IPW) during training, or position-free inference (set `inv_position` = 0 at serving time and use it only for debiasing).

Distribution shift (temporal drift). News topic distributions shift over time (elections, sports seasons, pandemics). The LightGBM model trained on 6-month-old data will underweight emerging topics. The FAISS index reflects old article embeddings and the BM25 vocabulary is frozen at index time. *Fix:* scheduled daily retraining (or online learning with a sliding window), and incremental FAISS index updates for newly published articles.

Multi-language degradation. Our single `paraphrase-multilingual-MiniLM-L12-v2` model handles 50+ languages but was primarily evaluated on English and Danish. Performance on under-represented languages (e.g., Bengali, Swahili) may drop significantly. *Fix:* language-specific fine-tuned models, or language-detection routing to specialised encoders.

12.2. System-Level Production Failures

FAISS GPU OOM during index rebuild. Adding new articles requires rebuilding the FAISS index (or incremental `add()`). At $10\times$ corpus ($1.3\text{M articles} \times 384 \times 4 \text{ B} = 2 \text{ GB}$), rebuilding requires allocating a second GPU index in parallel (for zero-downtime swap), requiring 4 GB VRAM simultaneously. A T4 with 15 GB handles this; a 16 GB A100 is the minimum safe margin at $10\times$. *Risk:* if the old index is not freed before the new one is built, GPU OOM kills the serving process.

BM25 cold-start on new articles. Newly published articles are not in the inverted index until it is rebuilt. Between index rebuilds (e.g., every 15 min), BM25 has zero recall for articles published in that window. *Fix:* real-time indexing with a write-ahead log (e.g., Elasticsearch for the BM25 tier, FAISS for the dense tier).

Popularity map staleness. The popularity map is built from training impressions and updated only at model-retraining time. During a breaking news event, an article can go from 0 to 500K clicks in 30 minutes. The stale map under-ranks it by $\log(1+0) - \log(1+500K) \approx -13$ in feature space. *Fix:* stream click events into a real-time popularity counter (Redis INCR), refreshed every 60 s.

Cascading failure: retrieval timeout. If FAISS or BM25 takes >50 ms (e.g., index rebuild, GC pause), the re-ranker receives 0 candidates and falls back to a default list. Without a circuit breaker, a slow FAISS call can cascade: pending requests pile up in the feature extraction queue, exhausting CPU threads and causing a full service timeout. *Fix:* per-stage timeouts (FAISS: 30 ms, feature extraction: 40 ms, LightGBM: 10 ms) with graceful fallback to popularity-based ranking.

Training–serving skew. Our training pipeline runs as a batch offline job. If the preprocessing logic (tokenisation, embedding normalisation, feature scaling) differs between the offline trainer and the online serving code, predictions will be systematically wrong — potentially for all users without any error log. *Fix:* shared featurisation library called identically at train and serve time, with a shadow-mode comparison test on every model deployment.

12.3. Adversarial / Edge-Case Failures

Click-farming / fake engagement. If a publisher inflates `log_popularity` via bot clicks, our model will rank their articles disproportionately highly. *Fix:* anomaly detection on per-article click rate; filter bots by session duration and click velocity.

Impression-list poisoning. An adversarial publisher could embed high-frequency keywords in subtitles to inflate BM25 recall. Our Jaccard `lex_overlap` feature would then be artificially high. *Fix:* normalise `lex_overlap` by document length; flag articles with anomalous keyword density.

History manipulation. If users click on articles purely to manipulate their future recommendations (e.g., clicking sports to get sports recommendations for someone else using the same account), the user embedding becomes inconsistent. *Fix:* session-level isolation (separate history per device/session), anomaly detection on click sequences.

13. Reproducibility

```

1 git clone https://github.com/imchaitanya0/news-retrieval-system
2 cd news-retrieval-system
3 python -m src.data.build_pipeline
4 python -m src.retrieval.bm25 --dataset mind --split val
5 python -m src.retrieval.semantic --dataset mind --split val
6 python -m src.ranking.train_lgbm --dataset mind
7 python -m src.evaluation.evaluate --dataset mind --strategy lgbm

```

All large files (embeddings, models, parquets) are excluded via `.gitignore`. The Kaggle notebook (`notebooks/final.ipynb`) handles download and extraction end-to-end.

14. Conclusion

We built a production-grade two-stage news recommendation system on MIND and EB-NeRD. Our LightGBM re-ranker with 8 behavioural features outperforms NRMS by **+4.65 pp nDCG@10** on MIND and **+2.73 pp** on EB-NeRD, with all gains statistically significant (95% bootstrap CI excludes zero). The pipeline processes MIND val (376K impressions) at

p99 = 26.8 ms, supporting 373 QPS on a single T4 — well within a 100 ms SLA. Key engineering contributions: (1) `scipy.sparse` BM25 bypass (9 h → 14 min); (2) GPU FAISS with CPU serialisation; (3) chunked LightGBM test inference (prevents 30 GB OOM); (4) GPU chunk-size increase 256 → 4096 (Cell 19: 100 min → 40 min).

References

- [1] F. Wu, Y. Qiao, J.-H. Chen, C. Wu, T. Qi, J. Lian, D. Liu, X. Xie, J. Gao, W. Wu, and M. Zhou. MIND: A large-scale dataset for news recommendation. *ACL 2020*, pages 3597–3606.
- [2] C. Wu, F. Wu, S. Ge, T. Qi, Y. Huang, and X. Xie. Neural news recommendation with multi-head self-attention. *EMNLP-IJCNLP 2019*, pages 6389–6394.
- [3] J. Kruse et al. EB-NeRD: A large-scale dataset for news recommendation. *ACM RecSys Challenge 2024*.
- [4] C. J. Burges. From RankNet to LambdaRank to LambdaMART: An overview. *Microsoft Research TR MSR-TR-2010-82*, 2010.

A. Engineering Bug Reference

All bugs are documented in `TECHNICAL_REFERENCE.md` in the repo.

§7.1 FAISS GPU serialisation: use `faiss.index_gpu_to_cpu()` before `write_index()`.

§7.2 PyTorch VRAM fragmentation: `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`.

§7.11 BM25 JAX/Numba bypass: raw `scipy.sparse` `matmul` replaces `bm25s.retrieve()`.

§7.13 LightGBM test OOM: chunked `inference()` with `behaviors.slice(start, 50000)`.